Experiment 3: Draw class diagram and use case diagram using any open source tool

1. **Use Case:**

Description:

A Use Case Diagram is a behavioral UML (Unified Modeling Language) diagram used during the requirements and design phases of software development. It provides a high-level visual summary of how users (actors) interact with a system to achieve specific goals, capturing system requirements and scope without exposing complex internal implementation details.

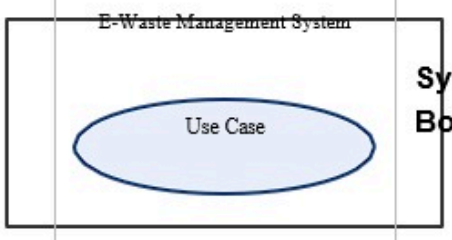
The diagram defines the functional scope of the **E-Waste Lifecycle Portal** enclosed within a system boundary rectangle. It models the core operations carried out by three primary actors:

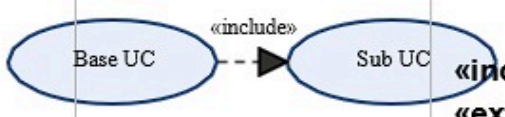
- **Donor / Resident:** Can initiate the Request E-Waste Pickup use case (which automatically triggers authentication) and track ongoing requests using Track Request Status.
- **Logistics Driver:** Interacts with the system to update and view Track Request Status and execute Assess & Log Scrap Material upon collecting electronic waste.
- **Compliance Auditor:** Responsible for triggering the Issue Compliance Certificate use case once processing meets environmental standards.

The diagram also specifies two key UML relationships:

- **<<include>> Relationship:** The Request E-Waste Pickup use case unconditionally includes Authenticate Account, meaning identity verification is mandatory whenever a pickup is requested.
- **<<extend>> Relationship:** The Flag Hazardous Components use case conditionally extends Assess & Log Scrap Material, meaning hazardous material flagging occurs only when specific hazardous items are identified during assessment.

Notations Used

Symbol	Notation	Meaning	Example use in this diagram
	**Actor**	A stick figure representing a role played by a user or an external system that interacts with the system being modelled.	Resident/Consumer, Recycling Facility, System Admin.
	**Use Case**	An oval representing a discrete piece of functionality or goal that the system provides to an actor.	Register/Login, Schedule Waste Pickup, Track Pickup Status, Redeem Rewards, Generate Recycling Reports, Manage Users, Update Pickup Status, Log Recycled Material & Weight.
	**System Boundary**	A labelled rectangle enclosing all the use cases, marking the scope of the system being modelled and separating it from its external actors.	The outer box labelled "E-Waste Management System" that contains all nine use cases.

Symbol	Notation	Meaning	Example use in this diagram
	**«include» / «extend»**	Dashed arrows between use cases: «include» means the base use case always triggers the included one; «extend» means the extending use case runs only under certain conditions.	Schedule Waste Pickup «include» Send Notifications; Verify Recycling Facility «extend» Log Recycled Material & Weight; Issue Recycling Rewards «extend» Redeem Rewards / Update Pickup Status.

Advantages

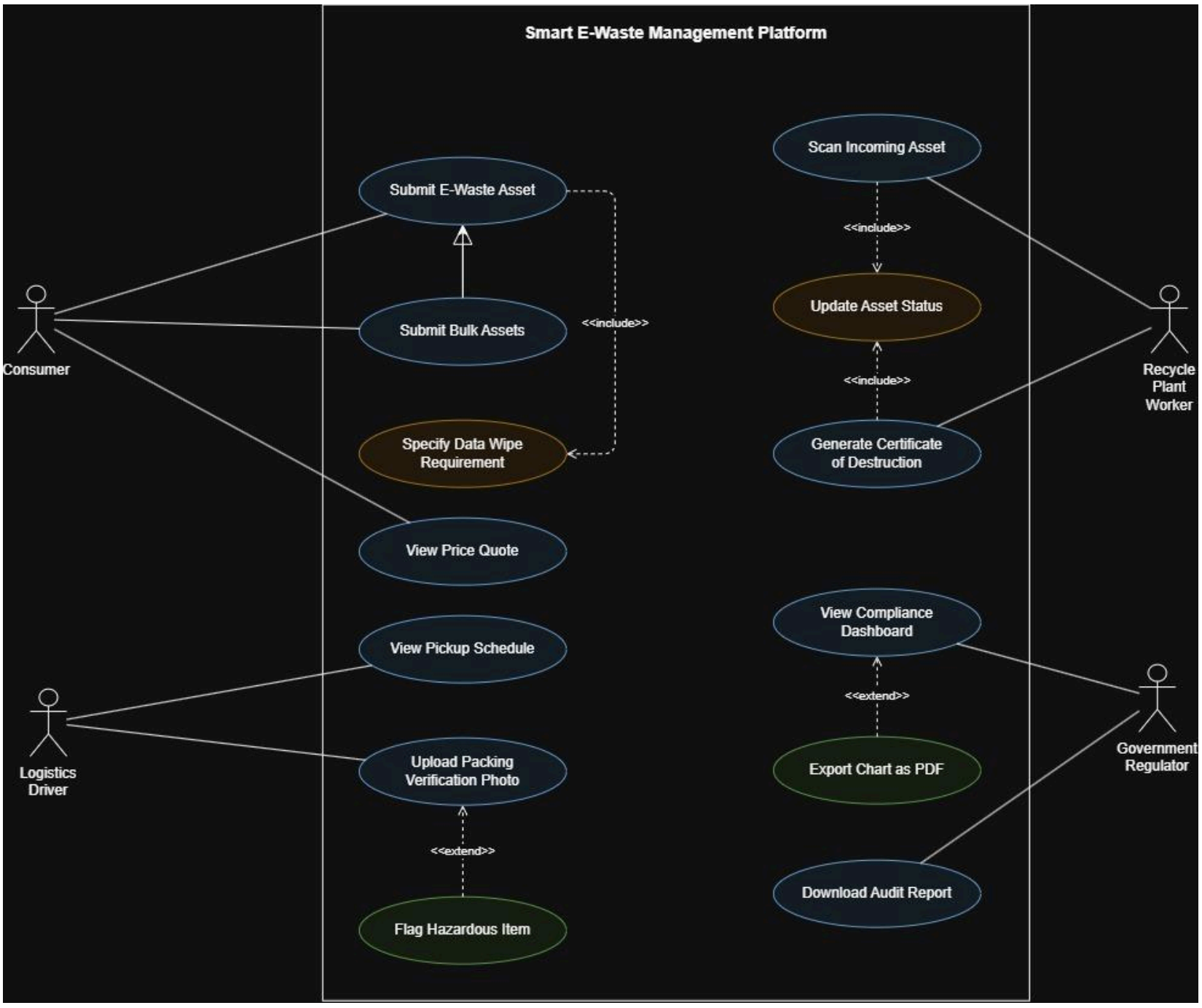
- **User-Centric Requirements:** Focuses directly on what actors need to accomplish, making it an effective tool for discovering and validating functional requirements.
- **Clear System Boundary:** Clearly differentiates between internal application features and external users or systems.
- **Communication Bridge:** Acts as an intuitive visual aid between non-technical clients/stakeholders and technical development teams.
- **Relationship Mapping:** Explicitly separates compulsory steps (<<include>>) from conditional or optional steps (<<extend>>).

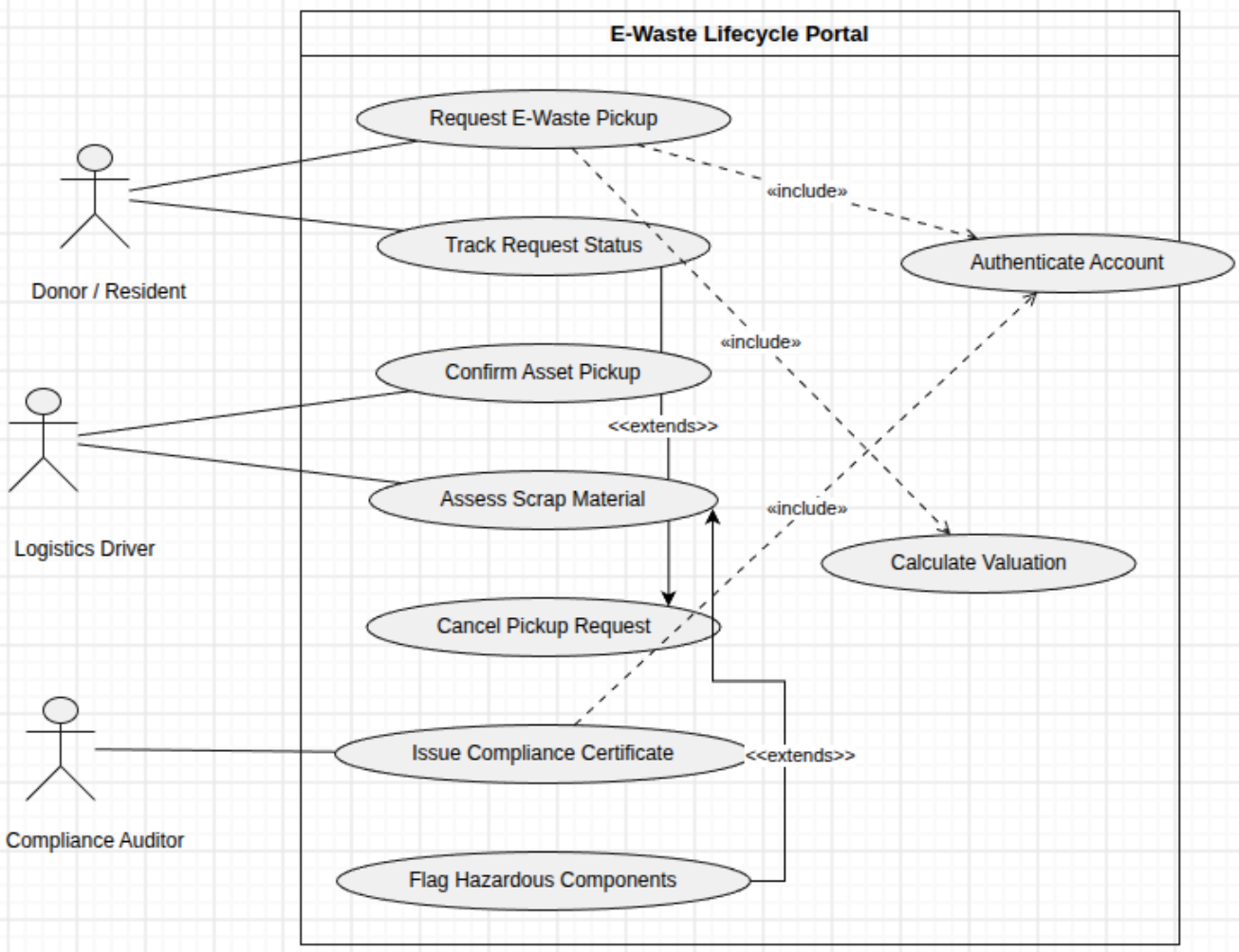
Disadvantages

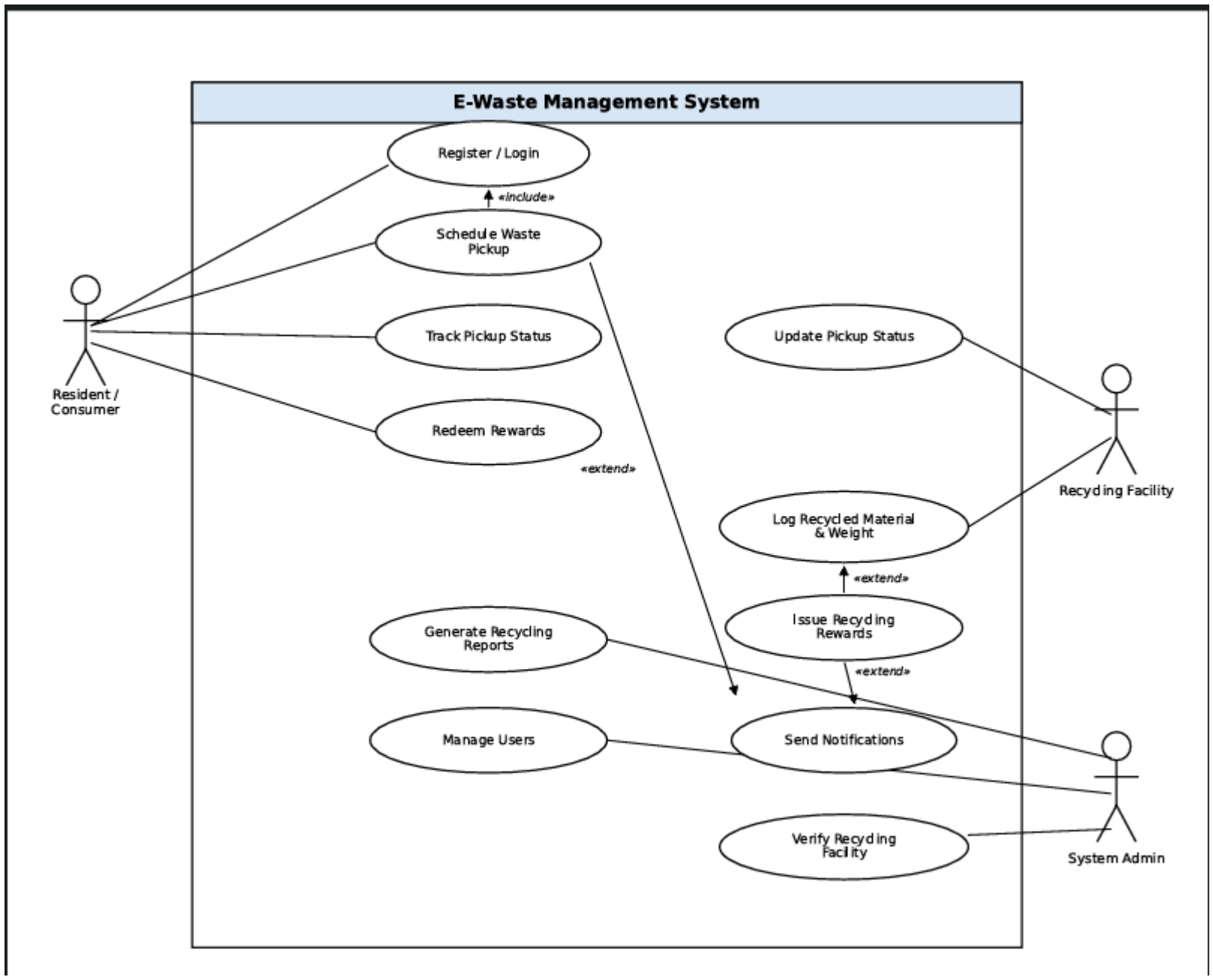
- **No Execution Sequence:** Does not illustrate the step-by-step order, timing, or lifecycle progression in which use cases are executed.

- **Lacks Detailed Logic:** Omits specific validation rules, input data formats, error handling paths, or algorithmic calculations.
- **Risk of Overcomplication:** Overuse of <<include>> and <<extend>> relationships can make large use case models difficult to interpret and maintain.

Output:




E-Waste Lifecycle Portal
Request E-Waste Pickup
Track Request Status
Confirm Asset Pickup
Assess Scrap Material
Cancel Pickup Request
Issue Compliance Certificate
Flag Hazardous Components
Calculate Valuation
Authenticate Account
«include»
«include»
«include»
<<extends>>
<<extends>>
Donor / Resident
Logistics Driver
Compliance Auditor


E-Waste Management System
Register / Login
«include»
Schedule Waste Pickup
Track Pickup Status
Redeem Rewards
«extend»
Resident / Consumer
Update Pickup Status
Recyding Facility
Log Recycled Material & Weight
«extend»
Issue Recyding Rewards
«extend»
Generate Recycling Reports
Manage Users
Send Notifications
Verify Recyding Facility
System Admin

2. Class Diagram:

Description:

A class diagram is a structural UML diagram that models the static design of a system: the classes that make it up, the data (attributes) and behaviour (operations) each class owns, and how the classes are related. In this diagram, the E-Waste Management System is modelled with the following classes:

Consumer (requests pickups and owns devices),

ElectronicDevice (can be marked as e-waste),

CollectionCenter (logs intake of devices),

Transporter (assigns routes and delivers e-waste),

RecyclingFacility (processes e-waste into materials),

RefurbishmentCenter (restores reusable devices),

Material (recovered from recycling and used to make new products),

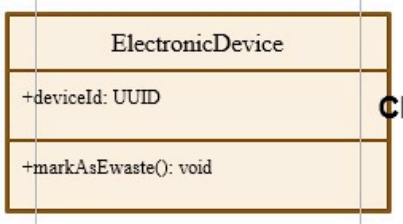
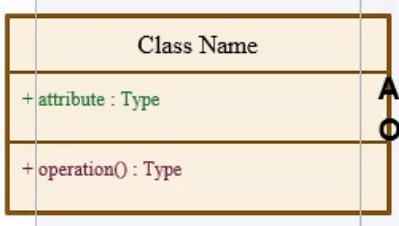
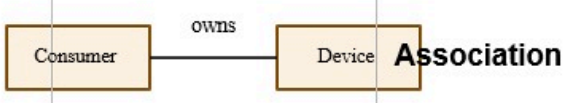
RecycledProduct (produced from materials and released to market),

DisposalCertificate (issued once processing is verified) and

ComplianceAuthority (audits and regulates the process by validating certificates).

Each class box lists its attributes (e.g. Consumer has *id: UUID* and *name: String*) and operations (e.g. Consumer has *requestPickup(deviceId): void*). Labelled association lines connect the classes to describe their real-world relationships, such as Consumer → ElectronicDevice ("owns"), RecyclingFacility → Material ("recovers"), Transporter → CollectionCenter ("delivers intake"), RecyclingFacility → DisposalCertificate ("issues") and ComplianceAuthority → DisposalCertificate ("validates").

Notations Used

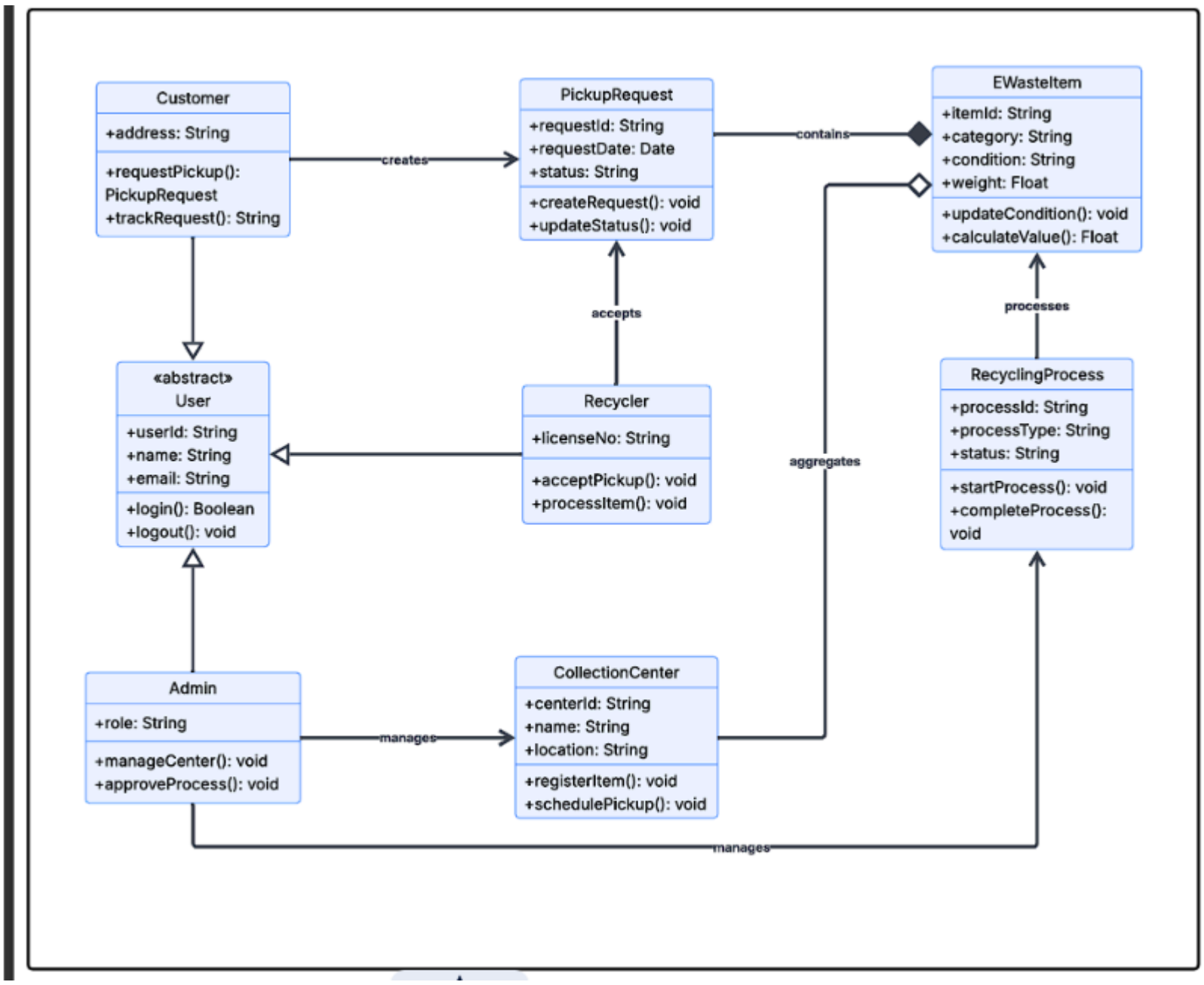
Symbol	Notation	Meaning	Example use in this diagram
	**Class**	A rectangle divided into three compartments: the class name, its attributes, and its operations/methods.	ElectronicDevice, Consumer, CollectionCenter, Transporter, RecyclingFacility, RefurbishmentCenter, Material, RecycledProduct, DisposalCertificate, ComplianceAuthority.
	**Attribute / Operation**	Attributes (data fields, shown with a type) sit in the middle compartment; operations (behaviours, shown with parameters/return type) sit in the bottom compartment.	Attributes such as deviceId: UUID, name: String; operations such as markAsEwaste(): void, processEwaste(): void, verify(): bool.
	**Association**	A line connecting two classes, usually labelled with a verb phrase, to show that instances of one class are related to instances of the other.	owns, recovers, assigns pickup, delivers intake, delivers e-waste, sends e-waste, sends reusable, refurbishes, produces, used in, issues, validates, regulates, submits pickup.

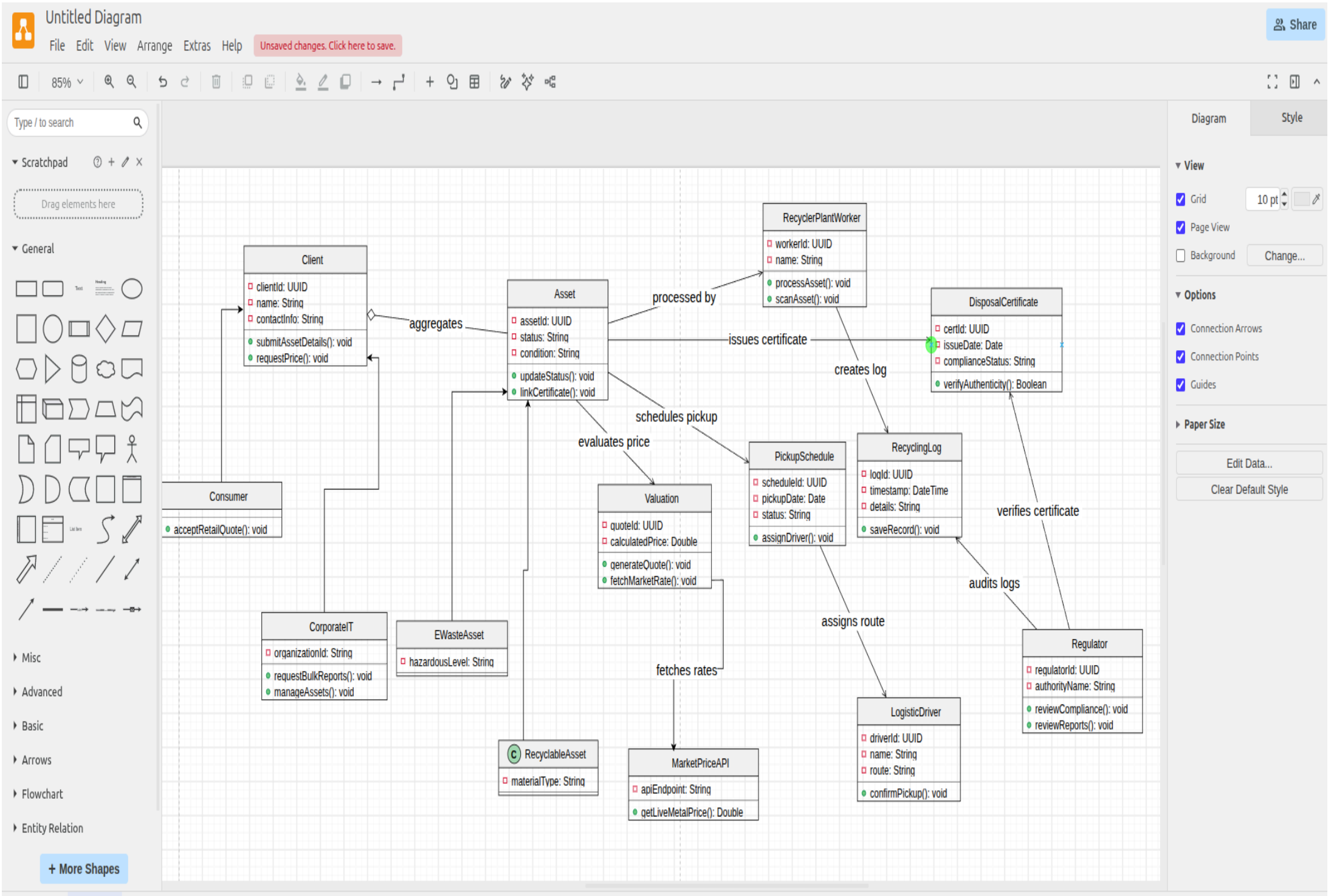
Advantages

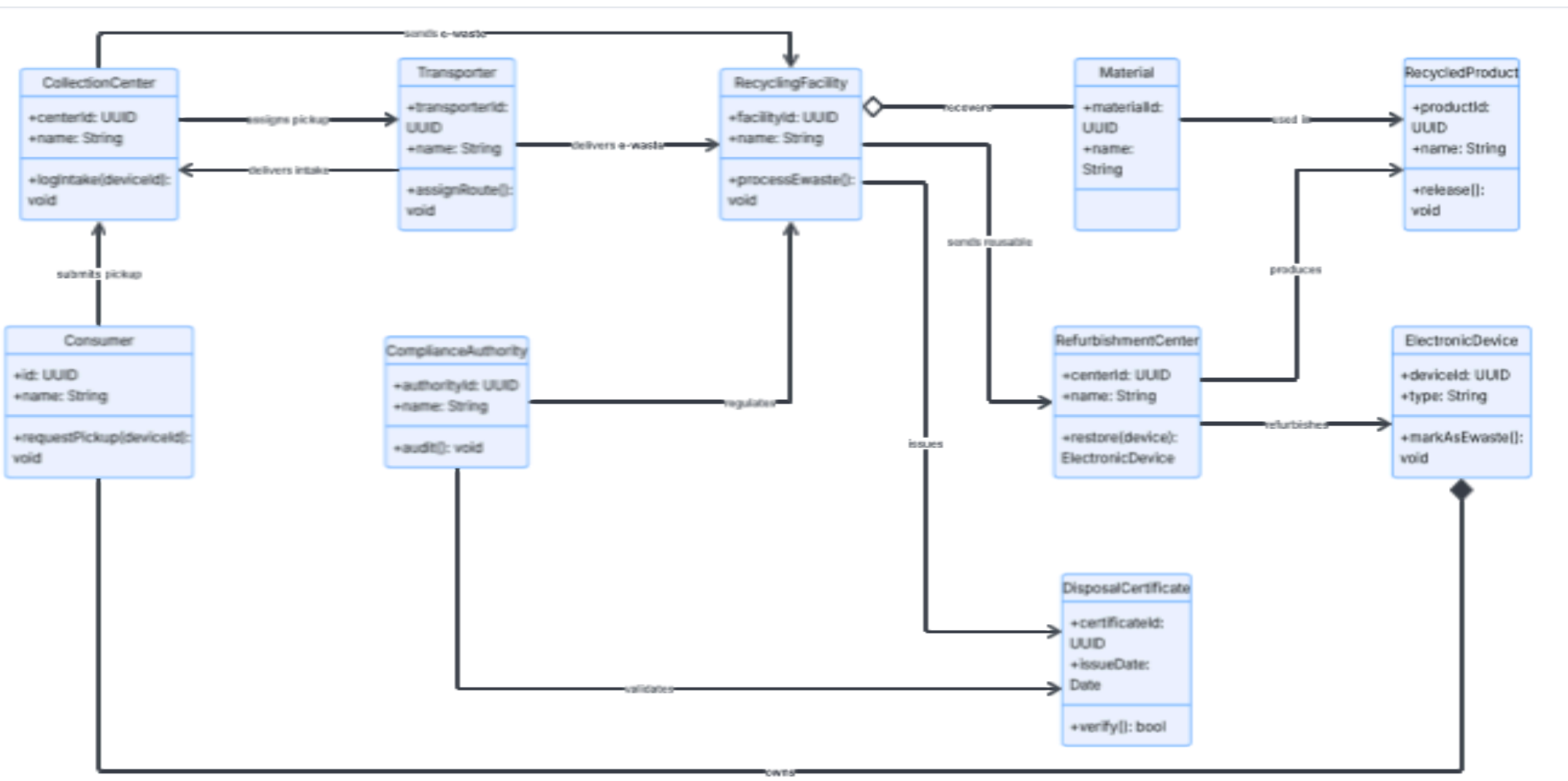
- **Blueprint for Implementation:** Directly maps to Object-Oriented Programming (OOP) code structures, making database schemas, classes, variables, and method signatures easy to implement.
- **Encapsulation & Architecture:** Clearly specifies data hiding and access levels using public and private visibility modifiers.
- **Modular Reusability:** Models inheritance hierarchies (Client->CorporateIT/Consumer), promoting code reuse and modular system maintenance.
- **Clear Interaction Contracts:** Defines exactly which classes interact, send messages, or hold dependencies across disparate application tiers.

Disadvantages

- **Static Representation Only:** Does not show dynamic run-time execution, sequence of operations, message order, or state transitions over time.
- **Maintenance Overhead:** System refactoring requires manual structural updates across diagrams to prevent design drift.
- **High Complexity at Scale:** As domain entities, relationships, and helper utilities multiply, class diagrams can become dense and harder to read.


Customer
+address: String
+requestPickup(): PickupRequest
+trackRequest(): String
creates
PickupRequest
+requestId: String
+requestDate: Date
+status: String
+createRequest(): void
+updateStatus(): void
contains
EWasteItem
+itemId: String
+category: String
+condition: String
+weight: Float
+updateCondition(): void
+calculateValue(): Float
accepts
processes
«abstract»
User
+userId: String
+name: String
+email: String
+login(): Boolean
+logout(): void
Recycler
+licenseNo: String
+acceptPickup(): void
+processItem(): void
aggregates
RecyclingProcess
+processId: String
+processType: String
+status: String
+startProcess(): void
+completeProcess(): void
Admin
+role: String
+manageCenter(): void
+approveProcess(): void
manages
CollectionCenter
+centerId: String
+name: String
+location: String
+registerItem(): void
+schedulePickup(): void
manages


Untitled Diagram
File Edit View Arrange Extras Help
Unsaved changes. Click here to save.
Share
Type / to search
Scratchpad
Drag elements here
General
Misc
Advanced
Basic
Arrows
Flowchart
Entity Relation
+ More Shapes
Diagram
Style
View
Grid
10 pt
Page View
Background
Change...
Options
Connection Arrows
Connection Points
Guides
Paper Size
Edit Data...
Clear Default Style
Client
clientId: UUID
name: String
contactInfo: String
submitAssetDetails(): void
requestPrice(): void
Asset
assetId: UUID
status: String
condition: String
updateStatus(): void
linkCertificate(): void
RecyclerPlantWorker
workerId: UUID
name: String
processAsset(): void
scanAsset(): void
DisposalCertificate
certId: UUID
issueDate: Date
complianceStatus: String
verifyAuthenticity(): Boolean
aggregates
processed by
issues certificate
creates log
schedules pickup
evaluates price
Consumer
acceptRetailQuote(): void
Valuation
quoteId: UUID
calculatedPrice: Double
generateQuote(): void
fetchMarketRate(): void
PickupSchedule
scheduleId: UUID
pickupDate: Date
status: String
assignDriver(): void
RecyclingLog
logId: UUID
timestamp: DateTime
details: String
saveRecord(): void
verifies certificate
audits logs
assigns route
CorporateIT
organizationId: String
requestBulkReports(): void
manageAssets(): void
EWasteAsset
hazardousLevel: String
fetches rates
Regulator
regulatorId: UUID
authorityName: String
reviewCompliance(): void
reviewReports(): void
RecyclableAsset
materialType: String
MarketPriceAPI
apiEndpoint: String
getLiveMetalPrice(): Double
LogisticDriver
driverId: UUID
name: String
route: String
confirmPickup(): void


CollectionCenter
+centerId: UUID
+name: String
+logIntake(deviceId[]): void
assigns pickup
delivers intake
Transporter
+transporterId: UUID
+name: String
+assignRoute(): void
delivers e-waste
RecyclingFacility
+facilityId: UUID
+name: String
+processEwaste(): void
receives
Material
+materialId: UUID
+name: String
used in
RecycledProduct
+productId: UUID
+name: String
+release(): void
submits pickup
sends reusable
produces
Consumer
+id: UUID
+name: String
+requestPickup(deviceId[]): void
ComplianceAuthority
+authorityId: UUID
+name: String
+audit(): void
regulates
RefurbishmentCenter
+centerId: UUID
+name: String
+restore(device): ElectronicDevice
refurbishes
ElectronicDevice
+deviceId: UUID
+type: String
+markAsEwaste(): void
issues
DisposalCertificate
+certificateId: UUID
+issueDate: Date
+verify(): bool
validates
owns